

Asistente de Originación Crediticia PyME

Análisis del enunciado, stack tecnológico
y plan de desarrollo por módulos

Examen práctico de AI Engineer · Gerencia de Innovación
Documento preparatorio · 21 de agosto de 2026

Dato	Valor
Fecha máxima de entrega	Martes 25 de agosto de 2026, 11:00 p. m.
Tiempo disponible	Aproximadamente 4 días naturales
Repositorio local	D:\Escritorio\ProyPersonal\asistente-originacion-pyme
Modelo de ramas	GitFlow, una rama de característica por módulo
Módulos planificados	23 módulos en 7 bloques

1. Qué se pide, en una página

El examen tiene **dos mitades independientes** que se califican por separado y ambas son obligatorias: un cuestionario técnico escrito y una prueba práctica de software.

1.1 Cuestionario técnico

Son 30 preguntas de razonamiento, con un máximo de 10 líneas por respuesta (12 líneas en la sección 4.F). No se evalúa la extensión sino los trade-offs, y hay que poder defender cada respuesta en una sesión en vivo. Se distribuyen así:

Sección	Preguntas	Temas
1. Bases de datos	3 (1.1–1.3)	Índices HNSW frente a IVFFlat con filtrado por tenant, idempotencia y aislamiento transaccional en herramientas de agente, y precálculo de indicadores frente a auditoría.
2. Backend	6 (2.1–2.6)	Endpoint de chat con streaming, superficie de riesgo de una herramienta que lee clientes, gestión de memoria y recolector de basura de V8, modelo de concurrencia de Node bajo carga mixta CPU/E-S, costo real de lanzar excepciones, y aritmética decimal en cuotas.
3. Frontend	3 (3.1–3.3)	SSE frente a WebSockets, degradación de React ante 40 tokens por segundo de streaming, y modelado del estado de una entidad que el agente construye progresivamente.
4.A–4.E Inteligencia artificial	15 (4.1–4.15)	Métricas con clases desbalanceadas, cuándo no usar un LLM, diseño de herramientas, multiagente frente a agente único, auditabilidad, fragmentación y recuperación, evaluación, control de costos, deriva silenciosa, y aprendizaje por refuerzo aplicado.
4.F Fronteras de la IA	3 de 12 a elegir	Investigación guiada. Exige al menos una fuente técnica primaria por respuesta. Una negativa bien fundamentada es una respuesta válida.

1.2 Prueba práctica

Construir un asistente que preanalice solicitudes de crédito PyME, aplique las políticas de crédito vigentes y emita una **recomendación** de dictamen citando la política que la sustenta. El enunciado insiste en un principio que condiciona toda la arquitectura:

Principio rector del punto 5.1

El sistema **no sustituye al analista**: produce insumos verificables para su decisión. El enunciado exige explícitamente que esto se vea reflejado en la arquitectura, no solamente en el README. En la práctica significa que el LLM nunca calcula un número ni decide en firme: propone, y el código verifica, cita y bloquea.

Sobre esa base, el sistema debe: calcular cinco indicadores financieros en código con decimal exacto, consultar un corpus de políticas y citarlo de forma verificable, ejecutar un agente con cinco herramientas mínimas, producir una salida estructurada validada por esquema, aplicar cinco guardarraíles fuera del prompt, dejar traza completa de cada ejecución y exponer una interfaz en React con chat en streaming, panel de dictamen en vivo, confirmación humana y una vista de métricas.

2. Entregables y criterios de aceptación

El enunciado enumera siete entregables. Esta tabla los cruza con lo que el plan produce y con el módulo responsable.

#	Entregable exigido	Dónde queda	Módulo
1	Repositorio, indicando lenguaje de backend, framework de agentes y frontend	Este repositorio, declarado en el README	M0, M20
2	Video mostrando funcionamiento y decisiones técnicas	Grabación local, no requiere despliegue	M23 (guion)
3	Cuestionario técnico contestado	docs/CUESTIONARIO.md	M21, M22
4	README de decisiones, con justificación de la estrategia de acceso al corpus	docs/DECISIONES.md	M20
5	Archivo JSON de políticas extendido a 25 o más	data/politicas.json	M3
6	Script y resultados de los 10 casos de evaluación	apps/api/src/eval/	M18
7	Bitácora de aprendizaje, máximo una página	docs/BITACORA.md	M20

2.1 Requisitos que se evalúan de forma binaria

Hay exigencias donde no existe cumplimiento parcial. Conviene tratarlas como criterios de aceptación duros y verificarlas con una prueba automatizada:

- **Cálculo en decimal exacto.** Los cinco indicadores del 5.3.1 se calculan en código con el tipo decimal del lenguaje. Nunca en punto flotante binario y nunca por el LLM.
- **Restricción en base de datos para G3.** El tope de monto_recomendado debe existir como restricción de la base de datos, no solo como validación en la aplicación. Una validación en el servicio no cumple el requisito.
- **Cita verificable en toda decisión.** El texto_literal de cada cita tiene que existir realmente en el corpus, validado en código. Sin cita verificable la decisión se fuerza a ESCALADO_A_COMITE.
- **Camino explícito de fallo en la salida estructurada.** El enunciado dice literalmente que reintentar a ciegas no es una respuesta aceptable, y que hay que documentar por qué el manejo elegido es mejor.
- **Distribución exacta de los 10 casos de evaluación.** Tres de aprobación, tres de rechazo por motivos de política distintos, dos de escalamiento (uno por monto y uno por ausencia de política aplicable) y dos adversariales.
- **Datos sintéticos con trampas incluidas.** Al menos 200 solicitudes, de las cuales 3 o más llevan intentos de manipulación en destino_fondos y 5 o más tienen datos financieros incompletos o inconsistentes.
- **Corpus con ambigüedad deliberada.** Al menos 25 políticas, con dos excepciones que modifican parcialmente una regla anterior y una situación no cubierta por ninguna política.

Punto extra: elegir uno solo

El apartado 5.4 advierte que no se intenten varios. La recomendación de este plan es **reordenamiento de resultados (reranking) con evidencia medida**, porque es el único de los cinco que además cobra los puntos extra del apartado 5.3.2 por comparar dos estrategias de acceso al corpus con datos. Las alternativas descartadas son servidor MCP, multimodal sobre estados financieros escaneados, memoria persistente entre sesiones y modelo como evaluador.

3. Tecnologías: qué impone el enunciado y qué queda a criterio

Muy poco es obligatorio. Esta es la separación exacta entre lo impuesto y lo elegible, con la decisión tomada para cada capa.

3.1 Lo que el enunciado impone

Capa	Restricción textual del enunciado
Frontend	Obligatorio: React (TypeScript o JavaScript) o Flutter en su canal web. No hay tercera opción. El manejo de estado y las herramientas de construcción son libres.
Proveedor de LLM	Obligatorio: OpenRouter con modelos gratuitos. No se contempla usar OpenAI o Anthropic directamente.
Framework de agentes	Prohibido: toda la familia LangChain y LangGraph, incluidos sus portes. El enunciado explica por qué: se evalúa el control sobre el ciclo de ejecución del agente, el contrato de las herramientas y el contenido exacto del contexto.
Tipo decimal	Obligatorio: decimal exacto por lenguaje. En Node.js, <code>decimal.js</code> o enteros en centavos.
Validación de salida	Obligatorio: validación por esquema antes de persistir. En Node.js, Zod o equivalente.
Guardarraíles	Obligatorio: G1 a G5 implementados fuera del prompt , y G3 con respaldo en la base de datos.

3.2 Lo que queda libre y la decisión tomada

Capa	Decisión	Por qué
Backend	Node.js 24 + TypeScript, servidor Fastify	El enunciado permite seis lenguajes y aclara que evalúa el criterio de diseño, no el lenguaje. Node.js comparte tipos con el frontend por Zod y hace de las preguntas 2.3 y 2.4 del cuestionario una defensa sobre código real, no hipotética.
Framework de agentes	Llamada directa al SDK del proveedor: paquete <code>openai</code> apuntado a OpenRouter, con ciclo de ejecución escrito a mano	El enunciado dice que llamar directamente al SDK siempre es válido si se justifica en el README, y que el framework recomendado no otorga puntos por sí mismo. Mastra resolvería el bucle, pero precisamente el bucle, el criterio de parada, el tope de costo y el contenido exacto del contexto son lo que se defiende en vivo. Escribirlo elimina una capa entre la decisión y su explicación.
Base de datos	PostgreSQL 16 con pgvector, en contenedor	El enunciado la recomienda explícitamente. pgvector habilita la parte vectorial de la recuperación híbrida y del punto extra sin añadir un servicio más.
Acceso a datos	Drizzle ORM sobre migraciones SQL escritas a mano	Las restricciones que exige G3 y la máquina de estados de G4 se escriben en SQL. Un ORM que genere las migraciones por inferencia estorbaría; se necesita control del DDL.
Frontend	React 19 + Vite + TypeScript, TanStack Query y Zustand	Query para el estado del servidor y Zustand para el estado efímero del streaming, que es donde nace el problema de rendimiento de la pregunta 3.2.
Estilos	Tailwind CSS v4 con sistema de tokens propio	Tema oscuro especializado y adaptable a móvil, construido sobre variables CSS para que el diseño sea consistente y no una colección de clases sueltas.
Transporte del chat	SSE con cancelación por AbortSignal	Es la respuesta que además se defiende en la pregunta 3.1: unidireccional, sobre HTTP, reconexión nativa, y suficiente mientras haya un solo espectador del flujo.
Embeddings	Locales, en proceso worker (@xenova/transformers)	OpenRouter no ofrece embeddings gratuitos fiables. Además convierte la pregunta 2.4 —cómputo intensivo de CPU conviviendo con conexiones en espera— en un problema real del proyecto.

4. Arquitectura propuesta

La arquitectura se organiza alrededor de una sola idea: **separar lo que el modelo propone de lo que el sistema afirma**. Todo lo determinista vive en código y entra al contexto ya calculado; el modelo solo redacta, selecciona políticas y propone una decisión, y esa propuesta atraviesa una capa de guardarraíles antes de tocar la base de datos.

4.1 Flujo de una evaluación

#	Etapas	Qué ocurre
1	Precálculo	Los cinco indicadores se calculan en código con decimal exacto y quedan persistidos. No se descubren llamando herramientas en cada turno: se inyectan como un bloque estable al inicio del contexto, que es también lo que exige el caché de prompt.
2	Contexto	Prompt de sistema versionado, bloque estable de indicadores, y destino_fondos encapsulado como dato no confiable en una sección delimitada que nunca se concatena con instrucciones.
3	Ciclo del agente	Bucle propio: el modelo pide herramientas, el despachador las ejecuta con contrato Zod, devuelve resultados como valor (nunca excepciones hacia el modelo) y el bucle corta por criterio de parada, tope de iteraciones o tope de costo.
4	Salida estructurada	El objeto Dictamen se valida contra Zod. Si falla, se repara con una pasada dirigida que recibe el error de validación concreto; si el presupuesto se agota, degrada a escalamiento.
5	Guardarrailes	G1 a G5 en código. Aquí una decisión puede ser forzada a ESCALADO_A_COMITE o rechazada la persistencia completa, independientemente de lo que dijera el modelo.
6	Persistencia	Escritura transaccional con clave de idempotencia generada por el servidor. La base de datos tiene la última palabra: sus restricciones rechazan lo que la aplicación dejara pasar.
7	Confirmación humana	Si G4 aplica, el dictamen queda en PENDIENTE_AUTORIZACION y no está en firme hasta que el analista confirme desde la interfaz.

4.2 Estructura del repositorio

asistente-origination-pyme/			
apps/			
api/	src/	config/	entorno, modelos, versiones de prompt
		db/	esquema Drizzle y migraciones SQL
		domain/	solicitudes, indicadores, politicas, dictámenes, metricas (sin LLM)
		agent/	loop, tools, prompts, guardrails, schema, repair (con LLM)
		observability/	registro de ejecuciones y costo
		http/	rutas Fastify y SSE
		eval/	10 casos y ejecutor
	web/	src/	tokens, primitivas, tema oscuro
		design-system/	chat, dictamen, solicitudes, metricas
		features/	cliente SSE, cliente API, formateadores
packages/			
shared/		esquemas Zod y tipos compartidos entre API y web	
data/		politicas.json y semillas sinteticas	
docs/		decisiones, cuestionario, bitacora, plan	

La frontera importante es domain/ frente a agent/: en domain/ no entra ninguna llamada a un LLM. Es lo que permite que el script de evaluación pruebe los cálculos y los guardarrailes sin gastar tokens, y es la traducción en código del principio de que el sistema no sustituye al analista.

5. Los cinco guardarraíles y cómo se implementan

El enunciado exige que estén **fuera del prompt**. Un guardarraíl escrito como instrucción al modelo no cuenta, porque el modelo puede ignorarlo. Cada uno se implementa como código que puede vetar el resultado.

ID	Exigencia	Implementación
G1	Ninguna decisión sin cita verificable; el texto literal debe existir en el corpus, validado en código	Normalización del texto (espacios, acentos, mayúsculas) y verificación contra el corpus. Si alguna cita no se encuentra, la decisión se fuerza a escalamiento y el motivo queda registrado. Se prueba con una cita deliberadamente inventada.
G2	Coherencia numérica: si un indicador del dictamen no coincide con la salida de calcular_indicadores, se rechaza la persistencia	Recomputación en decimal exacto y comparación estricta contra el objeto propuesto por el modelo. La comparación es sobre el valor decimal, no sobre la cadena formateada.
G3	Topes de monto con restricción a nivel de base de datos, no solo en la aplicación	Restricción CHECK sobre la tabla de dictámenes, con el monto solicitado y el tope de política aplicable materializados en la propia fila para que la restricción pueda evaluarlos. Se prueba con un INSERT que debe fallar.
G4	Autorización humana para montos mayores a Q250,000 o riesgo ALTO; sin confirmación explícita el dictamen no queda en firme	Estado PENDIENTE_AUTORIZACION forzado por trigger o restricción, más un endpoint de confirmación separado. La interfaz distingue visualmente propuesta reversible de efecto ya confirmado en el servidor.
G5	destino_fondos es entrada no confiable con intentos de manipulación; el agente no debe alterar su comportamiento por lo que diga	El campo nunca entra al prompt de sistema. Se encapsula en una sección delimitada y marcada como dato del solicitante, se le aplica un detector de instrucciones dirigidas al asistente, y las herramientas jamás toman parámetros derivados de él. Se prueba con las tres solicitudes contaminadas del conjunto sintético.

El argumento en contra de la propia solución

La pregunta 2.2 del cuestionario pide argumentar contra las propias mitigaciones. Vale la pena anotarlo desde ahora, mientras se implementa: G5 aísla la instrucción, pero no impide que el solicitante escriba en destino_fondos un texto que *parezca* evidencia legítima y sesgue la selección de políticas del modelo. Ese ataque sigue siendo viable, y admitirlo con precisión vale más que declarar el problema resuelto.

6. Plan de desarrollo por módulos

23 módulos cortos en 7 bloques. Cada módulo es una unidad cerrada con criterio de cierre verificable, se desarrolla en su propia rama de característica y se integra a develop antes de empezar el siguiente. El avance se dispara con el comando **continua**.

ID	Módulo	Contenido y criterio de cierre
Bloque A · Cimientos		
M1	Andamiaje del monorepo e infraestructura local	Monorepo pnpm, TypeScript estricto, Docker Compose con PostgreSQL y pgvector, variables de entorno, esqueletos de Fastify y Vite que arrancan.
M2	Esquema de base de datos y migraciones	Migraciones SQL versionadas, restricciones de G3 y G4 en la base de datos, estrategia de precálculo de indicadores con su invalidación, tablas de observabilidad.
M3	Corpus de políticas	Extensión del JSON a 25 o más políticas con dos excepciones parciales, una situación no cubierta y categorías discriminables. Cargador idempotente.
Bloque B · Dominio determinista		
M4	Núcleo financiero en decimal exacto	Cuota nivelada, tabla de amortización iterativa, política de redondeo declarada, reparto del residuo. Los cinco indicadores del 5.3.1.
M5	Generación de datos sintéticos	200 o más solicitudes con semilla fija, 3 con intentos de manipulación, 5 con datos inconsistentes, más histórico de dictámenes para las métricas.
M6	Recuperación de políticas y verificación de citas	Enrutamiento por categoría, recuperación léxica y vectorial, resolución de precedencia entre regla general y excepción, verificador de literalidad.
Bloque C · Agente		
M7	Ciclo de ejecución propio	Bucle escrito a mano, despacho de herramientas, criterio de parada, tope de iteraciones y de costo, prompts versionados en archivo.
M8	Herramientas del agente	Las cinco herramientas obligatorias con contrato Zod de entrada y salida, y errores devueltos como valor en lugar de excepciones hacia el modelo.
M9	Salida estructurada y camino de fallo	Esquema Dictamen en Zod, reparación dirigida con el error de validación como retroalimentación, presupuesto acotado y degradación a escalamiento.
M10	Guardarrailes G1 a G5	Implementación fuera del prompt, con una prueba por guardarraíl que demuestra el bloqueo efectivo.
M11	Persistencia idempotente y observabilidad	Escritura transaccional con clave generada por el servidor. Registro de sesión, versión de prompt, modelo, herramientas, tokens, latencia y costo.
Bloque D · API		
M12	API HTTP y streaming	Endpoints de solicitudes, dictámenes, confirmación y métricas. Chat por SSE con cancelación propagada hasta el proveedor.
Bloque E · Frontend		
M13	Sistema de diseño y shell responsive	Tema oscuro especializado, tokens de color y tipografía, primitivas de interfaz, navegación adaptable de escritorio a móvil.
M14	Chat en streaming con actividad del agente	Renderizado incremental sin bloquear el hilo principal, cancelación, y línea de tiempo de herramientas invocadas y fuentes consultadas.
M15	Panel de dictamen en vivo	Renderizado progresivo del objeto estructurado, citas con identificador y sección, nivel de confianza y confirmación explícita para G4.
M16	Bandeja y detalle de solicitudes	Listado filtrable con indicadores precalculados e historial de dictámenes por solicitud.
M17	Vista de métricas	Solicitudes procesadas por estado, monto promedio recomendado y tasa de escalamiento.
Bloque F · Evaluación y punto extra		

ID	Módulo	Contenido y criterio de cierre
M18	Banco de evaluación	Los 10 casos con la distribución exigida, script ejecutor, criterio de aprobación documentado y reporte de resultados.
M19	Punto extra: reranking con evidencia medida	Reordenamiento de fragmentos recuperados y comparación A/B contra la línea base, con métricas de precisión de citas.
Bloque G · Documentación y entrega		
M20	README de decisiones y bitácora de aprendizaje	Justificación de la estrategia de acceso al corpus y de la estrategia de precálculo. Bitácora de una página.
M21	Cuestionario, secciones 1 a 3	Bases de datos, backend y frontend: 12 respuestas de un máximo de 10 líneas cada una.
M22	Cuestionario, sección 4	Las 15 preguntas de 4.A a 4.E más tres de las doce de 4.F, con fuente técnica primaria en cada una.
M23	Cierre y entrega	Rama de publicación, etiqueta v1.0.0, verificación final de los siete entregables y guion del video.

7. GitFlow ligado al proceso

El modelo de ramas no es decorativo: hace que el historial cuente la misma historia que el plan. En la defensa técnica se puede señalar el commit exacto donde entró cada guardarraíl.

Rama	Nace de	Muere en	Regla
main	—	—	Estado entregable. Solo recibe merges desde publicación o corrección urgente, siempre etiquetados.
develop	main	—	Integración de módulos terminados. Base de toda rama de característica. Solo merges sin avance rápido.
feature/M<n>-<slug>	develop	develop	Exactamente un módulo del plan por rama. Se integra con merge sin avance rápido para conservar la frontera del módulo.
release/<version>	develop	main y develop	Congelación de alcance, últimos ajustes de documentación y etiqueta.
hotfix/<slug>	main	main y develop	Solo para lo que rompa un entregable después de una etiqueta.

Los commits siguen Conventional Commits en español con ámbito explícito, por ejemplo feat(guardrails): forzar escalamiento cuando la cita no es literal. El detalle completo está en docs/GITFLOW.md.

8. Lo que necesito de ti antes o durante el desarrollo

Esta es la parte que conviene leer con más atención. Hay cosas que no puedo hacer y otras que sí puedo hacer pero no debo hacer por ti.

8.1 Bloqueantes: sin esto el desarrollo se detiene

Qué	Por qué me bloquea	Qué tienes que hacer
Clave de API de OpenRouter	El enunciado impone OpenRouter como proveedor. No puedo crear cuentas ni registrarme por ti, y no debo manejar credenciales en texto plano.	Regístrate en <code>openrouter.ai</code> , genera una clave y pégala tú mismo en <code>apps/api/.env</code> como <code>OPENROUTER_API_KEY</code> . Yo dejo el <code>.env.example</code> preparado en M1 y el <code>.env</code> está en gitignore. Se necesita en M7 , no antes.
Docker Desktop en ejecución	Docker está instalado (versión 28.5.2) pero el motor no está corriendo : verifiqué y el pipe de Docker Desktop no responde. Sin él no hay PostgreSQL.	Abre Docker Desktop y espera a que el motor quede en verde. Se necesita en M1 , es decir, en el primer continua .

8.2 No bloqueantes, pero decides tú

Qué	Situación
Repositorio remoto en GitHub	Tu CLI gh ya está autenticado como <code>MarioCrV208</code> con permisos suficientes. Puedo crear el repositorio privado y publicar las ramas cuando me lo digas. Lo dejo privado a propósito: tú decides cuándo hacerlo público o dar acceso a los evaluadores.
Video de demostración	Es un entregable obligatorio y no puedo grabarlo. En M23 te preparo el guion con el recorrido, los puntos técnicos a mencionar y los tiempos sugeridos, pero la grabación y la voz son tuyas.
Envío del correo final	Los siete entregables van a seis direcciones de <code>genesisempresarial.com</code> . No envío correos en tu nombre. Te dejo todo listo y tú lo envías.
Datos personales del examen	La portada del cuestionario pide nombre del aspirante y teléfono. Los dejo en blanco para que los llenes tú.
Defensa técnica en vivo	El enunciado pide documentar cómo se usó IA para resolver la prueba y poder defender cada decisión. Redacto las respuestas del cuestionario, pero conviene que las revises antes de enviarlas: en la sesión en vivo te van a preguntar por ellas y tienen que ser tuyas de verdad.

Qué	Situación
Consulta por WhatsApp	Solo haría falta si quisiéramos un framework fuera de la tabla permitida. Con la decisión de llamar directamente al SDK del proveedor no hace falta consultar nada: el enunciado lo autoriza de forma explícita para los seis lenguajes.
Descarga del modelo de embeddings	La primera ejecución de M6 descarga un modelo local de unos 100 MB. Requiere internet y espacio en disco, nada más. Si prefieres evitarlo, hay un plan alternativo: con 25 a 40 políticas el corpus entero cabe en contexto y la parte vectorial pasa a ser opcional.

9. Riesgos y calendario

El riesgo principal es el calendario, no la técnica

Hoy es viernes 21 de agosto y la entrega vence el martes 25 a las 11 de la noche. Son cuatro días para 23 módulos más 30 respuestas de cuestionario más un video. El plan es alcanzable porque los módulos son cortos, pero no sobra tiempo: conviene no interrumpir la secuencia y dejar el punto extra (M19) como lo primero que se sacrifica si el calendario aprieta.

Riesgo	Impacto	Mitigación
Los modelos gratuitos de OpenRouter limitan peticiones por minuto y a veces degradan la calidad de las llamadas a herramientas	Alto	Configurar dos o tres modelos con cascada de reserva desde el inicio. El script de evaluación no depende del LLM para los cálculos ni los guardarraíles, así que buena parte se puede verificar sin gastar cuota.
La salida estructurada falla con modelos pequeños	Alto	Es justamente lo que M9 resuelve, y el enunciado lo exige. La reparación dirigida con el error de validación concreto se implementa antes que cualquier trabajo de interfaz.
El cuestionario se subestima: son 30 respuestas razonadas	Alto	Está partido en dos módulos (M21 y M22) y no se deja para el final absoluto. Las notas técnicas se van tomando durante la implementación, cuando el argumento está fresco.
La restricción de base de datos para G3 necesita datos de otra tabla	Medio	Un CHECK solo ve columnas de su propia fila. Se resuelve materializando el monto solicitado y el tope de política aplicable en la fila del dictamen. Está previsto en M2.
El diseño de interfaz consume más tiempo del previsto	Medio	El enunciado marca el diseño visual pulido como fuera de alcance. El sistema de diseño de M13 se construye una sola vez y las pantallas posteriores solo lo consumen.
Docker en Windows con pgvector	Bajo	Se usa la imagen oficial pgvector/pgvector, que ya trae la extensión compilada. Sin pasos manuales.

9.1 Sobre el diseño de interfaz que pediste

Vale la pena dejarlo por escrito porque hay una tensión con el enunciado. El punto 5.6 marca explícitamente como **fuera de alcance** el diseño visual pulido, la identidad de marca y el diseño adaptable avanzado. Pediste un tema oscuro altamente estilizado y adaptable a móvil, y se va a hacer, pero conviene entender qué significa eso aquí: no suma puntos por sí mismo y no debe costarle tiempo a lo que sí se califica.

Ahora bien, hay una parte de la interfaz que **no** es cosmética y que el enunciado destaca en un recuadro propio: la aplicación debe transmitir que hay un sistema inteligente trabajando activamente, en lugar de parecer un tablero de inteligencia de negocios. Eso significa comunicar progreso, acciones ejecutadas, fuentes consultadas, resultados parciales, nivel de confianza y qué decisiones requieren intervención humana, sin exponer razonamiento interno sensible. Ese requisito es de producto, no de estética, y es donde se concentra el esfuerzo de M14 y M15.

10. Qué pasa ahora

El repositorio local ya existe con GitFlow inicializado y este documento dentro. Escribe **continua** para arrancar con M1, el andamiaje del monorepo y la infraestructura local. Antes de ese primer **continua**, abre Docker Desktop.